

ZX-Calculus: The Game

A Visual and Interactive Introduction to Quantum Reasoning

Abdullah K

June 12, 2025

Introduction

ZX-calculus is a powerful graphical language used for reasoning about quantum processes. It represents quantum operations as diagrams, with two types of "spiders" — green Z -spiders and red X -spiders — as well as Hadamard gates (drawn as yellow squares). These diagrams can be simplified using a set of rewrite rules, analogous to algebraic identities in circuit theory. The formalism is complete for pure qubit quantum mechanics and enables simplification, optimization, and even verification of quantum circuits.

Diagram Example using TikZ

Below is a simple ZX-diagram consisting of a Z -spider connected to an X -spider via a Hadamard edge:



This structure can be simplified using the *color-change rule*, where the Hadamard flips the color of the spider it connects to.

ZX-Simplifier: The Game

ZX-Simplifier is a Python-based interactive puzzle game designed to help learners explore ZX-calculus through applied gameplay. Players are given a ZX-diagram and must apply valid rewrite rules to reduce it step by step, aiming to reach a fully simplified form within a limited number of moves.

Key Classes and Architecture

Node Class: Represents a node in the ZX-diagram with a type (Z , X , H , $WIRE_END$) and optional phase.

- `add_connection`, `remove_node`, `get_neighbors` for graph manipulation.

ZXDiagram Class: Maintains the node dictionary and handles structural operations like adding nodes and connections.

ZXRules Class: Implements the core rewrite rules:

- **Spider Fusion Rule:** Combines two connected spiders of the same type, adding their phases.
- **Identity Rule:** Removes phase-zero spiders with two connections.
- **Hadamard Color Change:** Eliminates Hadamard and flips spider color.

QuantumGame Class: Manages the gameplay loop:

- Initializes diagram
- Displays rules
- Applies chosen simplification rules
- Checks win condition

Sample Diagram Puzzle

The initial puzzle might look like:

WIRE_END -- Z(0) -- WIRE_END

This is solvable by applying the Identity Rule, connecting the two wire ends directly and deleting the identity spider.

Code Logic Overview

The Python implementation is modular and intuitive. Here's a high-level outline of how it works:

- Users interactively apply simplification rules to a graph-like structure.
- After each operation, the internal state of the diagram is updated.
- The game ends when the diagram reaches its most simplified form.

ZX-Calculus is a graphical language for reasoning about quantum circuits and computations. It simplifies quantum processes using intuitive diagrams made of spiders (nodes), Hadamard gates, and wires. Each element represents algebraic structures fundamental to quantum theory.

ZX-Calculus: The Game is an educational game that combines the ZX formalism with interactive simplification tasks to teach users how to rewrite and reduce quantum diagrams using core rules of the calculus.

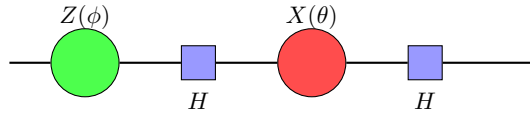
1 Why Learn Through a Game?

This game-based approach to ZX-calculus provides several educational and practical benefits:

- **Hands-on Intuition:** Learning through interaction helps build intuition around the abstract concepts of ZX-diagrams.
- **Rule Familiarity:** Players gain experience with core rewrite rules like spider fusion, identity removal, and Hadamard color change.
- **Quantum Reasoning:** Develops a deeper understanding of how graphical reasoning relates to quantum linear algebra and circuit simplification.
- **Visualization:** Provides visual feedback on simplification and correctness, reinforcing logical structures.

2 Example: A Simple ZX Diagram

Below is a TikZ representation of a simple ZX-diagram involving a green spider (Z), a red spider (X), and Hadamard gates:



This diagram can be simplified using rewrite rules such as:

- **Hadamard Color Change:** Converts between green and red spiders.
- **Spider Fusion:** Merges connected spiders of the same type.
- **Identity Removal:** Eliminates spiders with 0 phase and degree 2.

2.1 Understanding the ZX-Calculus: The Building Blocks

The **ZX-calculus** is a powerful and rigorous graphical language used to represent and reason about quantum circuits. It uses string diagrams (called **ZX-diagrams**) that visually represent quantum operations.

Generators: Spiders, Wires, and Hadamard Gates

- **Green Spiders (Z-spiders):** Represent operations in the computational basis and can carry a phase.
- **Red Spiders (X-spiders):** Represent operations in the Hadamard-transformed basis and also support phases.
- **Phases:** Real numbers in $[0, 2\pi)$; phase 0 is often omitted.

- **Wires:** Connect spiders; represent qubit flow.
- **Hadamard Gates (H):** Basis transformations; switch spider color.

Composition

- **Sequential:** Connecting outputs to inputs
- **Parallel:** Stacking vertically for different qubits

Only Topology Matters You can deform ZX-diagrams freely as long as connections are preserved—capturing the topological nature of quantum reasoning.

2.2 Game Mechanics: Rewrite Rules

The heart of ZX-calculus lies in its **graphical rewrite rules**, which preserve the meaning of the quantum operation.

Rule 1: Spider Fusion (Merge)

- Two same-colored connected spiders can fuse. Phases add.
- Example: $--Z(\phi_1)--Z(\phi_2)--$ becomes $--Z(\phi_1 + \phi_2)--$

Rule 2: Identity Simplification

- A spider with phase 0 and two connections acts as identity.
- Example: $--Node1--Z(0)--Node2--$ becomes $--Node1--Node2--$

Rule 3: Hadamard Color Change

- Hadamard switches $Z \leftrightarrow X$ and disappears.
- Example: $--H--Z(\phi)--$ becomes $--X(\phi)--$

2.3 Designing the Game: ZX-Simplifier

Game Overview A command-line game to simplify ZX-diagrams interactively using rewrite rules.

Gameplay Flow

1. View current diagram.
2. Choose and apply a rule.
3. Select nodes, apply transformation.
4. Repeat until simplified.

Win Condition Diagram becomes a straight wire: $\text{WIRE_START} \rightarrow \text{WIRE_END}$

Additional Rules

- Limited number of moves
- Only valid moves allowed
- Diagram must remain topologically valid

3 Advance Level Walkthroughs (Spoiler Alert!)

Below are the optimal steps for each level in the ZX-calculus simplification game. Use them as hints or a full guide to master the rules efficiently.

Level 1: The Simple Identity

Initial Diagram:

$$W(0) \text{ -- } Z(1, \phi = 0.00\pi) \text{ -- } W(2)$$

Optimal Moves (1):

1. Choose **Rule 2** (Identity Rule).
Enter ID of spider: 1

Result:

$$W(0) \text{ -- } W(2) \quad \checkmark (\text{Win!})$$

Level 2: Fusing Spiders

Initial Diagram:

$$W(0) \text{ -- } Z(1, \phi = 1.00\pi) \text{ -- } Z(2, \phi = 1.00\pi) \text{ -- } W(3)$$

Optimal Moves (2):

1. Choose **Rule 1** (Spider Fusion).
Enter ID of first spider: 1
Enter ID of second spider: 2
(Diagram becomes: $W(0) \text{ -- } Z(1, \phi = 2.00\pi) \text{ -- } W(3)$; note $2.00\pi \equiv 0.00\pi$)
2. Choose **Rule 2** (Identity Rule).
Enter ID of spider: 1

Result:

$$W(0) \text{ -- } W(3) \quad \checkmark (\text{Win!})$$

Level 3: Hadamard Challenges

Initial Diagram:

$$W(0) \text{ -- } Z(1, \phi = 0.00\pi) \text{ -- } H(2) \text{ -- } X(3, \phi = 0.00\pi) \text{ -- } H(4) \text{ -- } W(5)$$

Optimal Moves (4):

1. Choose **Rule 3** (Hadamard Color Change).
Enter ID of Hadamard node: 2
Enter ID of connected spider: 1
(*Diagram becomes: $W(0) \text{ -- } X(1, \phi = 0.00\pi) \text{ -- } X(3, \phi = 0.00\pi) \text{ -- } H(4) \text{ -- } W(5)$*)
2. Choose **Rule 1** (Spider Fusion).
Enter ID of first spider: 1
Enter ID of second spider: 3
(*Diagram becomes: $W(0) \text{ -- } X(1, \phi = 0.00\pi) \text{ -- } H(4) \text{ -- } W(5)$*)
3. Choose **Rule 3** (Hadamard Color Change).
Enter ID of Hadamard node: 4
Enter ID of connected spider: 1
(*Diagram becomes: $W(0) \text{ -- } Z(1, \phi = 0.00\pi) \text{ -- } W(5)$*)
4. Choose **Rule 2** (Identity Rule).
Enter ID of spider: 1

Result:

$$W(0) \text{ -- } W(5) \quad \checkmark (\text{Win!})$$